```python
import torch
import numpy as np
import onnx
import onnxruntime as ort
from ultralytics import YOLO
from onnxconverter_common import float16
```

Подключаю необходимые библиотеки.
1) Torch - для ML
2) Numpy - для работы с большим объёмом информации
3) onnx - по ТЗ, для перевода модели в формат .onnx и дальнейшего перевода (по необходимости) в другой формат
4) onnxruntime - среда исполнения для моделей формата .onnx
5) ultralytics - для использования встроенной в него модели YOLO
6) onnxconverter_common.float16 - модуль реализует преобразование типов данных в моделях ONNX из FP32 в FP16.

```python
yolo = YOLO("yolov8n.pt")
pt_model = yolo.model
```

Загружаем модель YOLO из файла "yolov8n.pt" (хранит обученную модель).
Извлекаем непосредственно нейронную сеть, используемую внутри объекта YOLO, в переменную pt_model.

```python
dummy_input = torch.randn(1, 3, 640, 640)  # batch x каналы x высота x ширина

onnx_output_path = "model_fp32.onnx"
torch.onnx.export(
    pt_model,
    dummy_input,
    onnx_output_path,
    export_params=True,  # сохраняем веса
    opset_version=13,    # версия ONNX
    do_constant_folding=True,  # оптимизация констант
    input_names=["images"],    # имя входного слоя
    output_names=["output"],   # имя выходного слоя
    dynamic_axes=None          # фиксированный размер батча
)

print("Модель успешно экспортирована в ONNX")
```

Генерируем с помощью библиотеки torch dummy_input - поддельное изображение с тремя каналами формата 640*640 пикселей.
В отдельную переменную сохраняем путь, где будет хранится экспортированная модель формата .onnx.
Далее экспортируем модель в формат .onnx, комментарии к аргументам на фото.

```
onnx_model = onnx.load(onnx_output_path)
onnx.checker.check_model(onnx_model)


transposed_model = float16.convert_float_to_float16(onnx_model)
onnx.save(transposed_model, "model_fp16.onnx")
print("Модель успешно конвертирована в FP16")
```

В отдельную переменную сохраняем экспортированную модель .onnx FP32.
Проверяем модель на корректность перед дальнейшим использованием.
Далее создаём переменную, чтобы сохранить модель, переведённую в FP16.
Используем функцию подключенной библиотеки.
Сохраняем эту модель в файл "model_fp16.onnx".

```
with torch.no_grad():
    pt_outputs = pt_model(dummy_input)  # Получаем tuple

if isinstance(pt_outputs, tuple):
    pt_output = pt_outputs[0].detach().cpu().numpy()  # берем первый выход
else:
    pt_output = pt_outputs.detach().cpu().numpy()
```

Так как мы тестируем модель, отключаем ненужное отслеживание градиентов для повышения эффективности вывода. Выполняем предсказание на фиктивном изображении. Приводим полученный результат к форме массива NumPy, подходящей для сравнения и анализа вне PyTorch-контекста.

```
ort_session = ort.InferenceSession("model_fp32.onnx")
outputs = ort_session.run(None, {"images": dummy_input.numpy()})
ort_output_fp32 = outputs[0]

ort_session_fp16 = ort.InferenceSession("model_fp16.onnx")
outputs_fp16 = ort_session_fp16.run(
    None, {"images": dummy_input.numpy().astype(np.float16)}
)
ort_output_fp16 = outputs_fp16[0]
```

InferenceSession. Создаёт сессию вывода для модели.
run(). Метод для запуска модели с определёнными входными данными. Первый аргумент: список индексов выходных слоёв (здесь None, значит вывод всех слоёв). Второй аргумент: словарь, где ключ соответствует имени входного слоя, а значение - входные данные.
Так как модель принимает изображение с именем входного слоя "images", мы передаём туда dummy_input.numpy().
Результатом выполнения является список массивов NumPy, соответствующих каждому выходу модели. Поскольку ожидается один выходной слой, выбираем нулевой элемент списка (outputs[0]).

Проворачиваем это с двумя видами моделей. FP32, FP16.

```
print("\nРезультаты:")
print("PyTorch output:", pt_output)
print("ONNX FP32 output:", ort_output_fp32)
print("ONNX FP16 output:", ort_output_fp16)
print("Разница FP32 vs PyTorch:", np.abs(pt_output -
ort_output_fp32).max())
print("Разница FP16 vs PyTorch:",
np.abs(pt_output.astype(np.float16) - ort_output_fp16).max())
```

Выводим результат. Это потери по предсказаниям.

Разница FP32 vs PyTorch: 0.0005187988
Разница FP16 vs PyTorch: 1.25

Но! Потери по времени:

```python
import time

# Функция для замера скорости
def benchmark(session, input_data, runs=100):
    # Прогрев
    session.run(None, {"images": input_data})

    # Замер времени
    start = time.perf_counter()
    for _ in range(runs):
        session.run(None, {"images": input_data})
    end = time.perf_counter()

    return (end - start) / runs


# Бенчмаркинг
fp32_time = benchmark(ort_session, dummy_input.numpy())
fp16_time = benchmark(ort_session_fp16, dummy_input.numpy().astype(np.float16))

print("\nСкорость выполнения:")
print(f"ONNX FP32: {fp32_time * 1000:.2f} мс/запрос")
print(f"ONNX FP16: {fp16_time * 1000:.2f} мс/запрос")
print(f"Ускорение: {fp32_time / fp16_time:.2f}x")
```

Итог: У меня установлена интегрированная видеокарта Intel Iris Xe Graphics, которая не поддерживает технологию CUDA от NVIDIA. Это означает, что я физически не могу использовать GPU для ускорения нейросетевых вычислений, так как CUDA является эксклюзивной технологией, работающей только на видеокартах NVIDIA. Моя система лишена специализированных аппаратных компонентов (тензорных ядер, CUDA-совместимых процессоров), необходимых для обработки параллельных вычислений в нейросетях. Все операции выполняются центральным процессором (CPU), который не оптимизирован для работы с форматом FP16, что добавляет лишние вычислительные этапы. Для реального ускорения мне потребуется дискретная видеокарта NVIDIA с поддержкой CUDA, либо переход на облачные решения с GPU-серверами.

Скорость выполнения:
ONNX FP32: 64.32 мс/запрос
ONNX FP16: 76.80 мс/запрос
Ускорение: 0.84x

Код:

```python
import torch
import numpy as np
import onnx
import onnxruntime as ort
from ultralytics import YOLO
```

```
from onnxconverter_common import float16

yolo = YOLO("yolov8n.pt")
pt_model = yolo.model

dummy_input = torch.randn(1, 3, 640, 640)  # batch x каналы x
высота x ширина

onnx_output_path = "model_fp32.onnx"
torch.onnx.export(
   pt_model,
   dummy_input,
   onnx_output_path,
   export_params=True,  # сохраняем веса
   opset_version=13,    # версия ONNX
   do_constant_folding=True,  # оптимизация констант
   input_names=["images"],    # имя входного слоя
   output_names=["output"],   # имя выходного слоя
   dynamic_axes=None          # фиксированный размер батча
)

print("Модель успешно экспортирована в ONNX")


onnx_model = onnx.load(onnx_output_path)
onnx.checker.check_model(onnx_model)


transposed_model =
float16.convert_float_to_float16(onnx_model)
onnx.save(transposed_model, "model_fp16.onnx")
print("Модель успешно конвертирована в FP16")

# Предикт через PyTorch
with torch.no_grad(): # не обучаем, а тестируем, поэтому
вырубаем запоминание метаданных
   pt_outputs = pt_model(dummy_input)  # Получаем tuple

if isinstance(pt_outputs, tuple):
   pt_output = pt_outputs[0].detach().cpu().numpy()  # берем
первый выход
else:
   pt_output = pt_outputs.detach().cpu().numpy()
```

```
ort_session = ort.InferenceSession("model_fp32.onnx")
outputs = ort_session.run(None, {"images":
dummy_input.numpy()})
ort_output_fp32 = outputs[0]


ort_session_fp16 = ort.InferenceSession("model_fp16.onnx")
outputs_fp16 = ort_session_fp16.run(
   None, {"images": dummy_input.numpy().astype(np.float16)}
)
ort_output_fp16 = outputs_fp16[0]


print("\nРезультаты:")
print("PyTorch output:", pt_output)
print("ONNX FP32 output:", ort_output_fp32)
print("ONNX FP16 output:", ort_output_fp16)
print("Разница FP32 vs PyTorch:", np.abs(pt_output -
ort_output_fp32).max())
print("Разница FP16 vs PyTorch:",
np.abs(pt_output.astype(np.float16) - ort_output_fp16).max())

import time

# Функция для замера скорости
def benchmark(session, input_data, runs=100):
   # Прогрев
   session.run(None, {"images": input_data})

   # Замер времени
   start = time.perf_counter()
   for _ in range(runs):
       session.run(None, {"images": input_data})
   end = time.perf_counter()

   return (end - start) / runs


# Бенчмаркинг
fp32_time = benchmark(ort_session, dummy_input.numpy())
fp16_time = benchmark(ort_session_fp16,
dummy_input.numpy().astype(np.float16))

print("\nСкорость выполнения:")
```

```
print(f"ONNX FP32: {fp32_time * 1000:.2f} мс/запрос")
print(f"ONNX FP16: {fp16_time * 1000:.2f} мс/запрос")
print(f"Ускорение: {fp32_time / fp16_time:.2f}x")
```

Выод:
PS C:\Users\89109\OneDrive\Рабочий стол\общая> python temp.py
Модель успешно экспортирована в ONNX
Модель успешно конвертирована в FP16

Результаты:
PyTorch output: [[[9.4053 12.668 18.91 ... 494.15 523.98 547.19]
 [4.5116 3.3552 3.6204 ... 590.54 592.35 572.34]
 [18.818 24.427 34.119 ... 290.38 231.12 187.17]
 ...
 [2.3677e-06 2.9139e-06 2.6998e-06 ... 2.3521e-06 1.9673e-06 2.8728e-06]
 [3.2953e-07 3.2826e-07 2.0368e-07 ... 1.25e-06 1.2461e-06 1.2948e-06]
 [2.4744e-06 2.0834e-06 1.499e-06 ... 2.313e-06 2.0552e-06 2.0545e-06]]]
ONNX FP32 output: [[[9.4053 12.668 18.91 ... 494.15 523.98 547.19]
 [4.5116 3.3552 3.6204 ... 590.54 592.35 572.34]
 [18.818 24.427 34.119 ... 290.38 231.12 187.17]
 ...
 [2.3842e-06 2.8908e-06 2.712e-06 ... 2.414e-06 1.9372e-06 2.8908e-06]
 [3.5763e-07 2.9802e-07 2.0862e-07 ... 1.2219e-06 1.2517e-06 1.2517e-06]
 [2.4736e-06 2.0564e-06 1.5199e-06 ... 2.2948e-06 2.0266e-06 2.0862e-06]]]
ONNX FP16 output: [[[9.375 12.656 18.906 ... 494 524 547.5]
 [4.5039 3.3594 3.6211 ... 590.5 592.5 572.5]
 [18.766 24.406 34.094 ... 290.5 230.88 187]
 ...
 [2.3842e-06 2.9206e-06 2.7418e-06 ... 2.3246e-06 2.0266e-06 2.9206e-06]
 [2.9802e-07 3.5763e-07 2.3842e-07 ... 1.3113e-06 1.3113e-06 1.3113e-06]
 [2.4438e-06 2.0862e-06 1.4305e-06 ... 2.3842e-06 2.0266e-06 2.0266e-06]]]

Разница FP32 vs PyTorch: 0.00079345703
Разница FP16 vs PyTorch: 1.1875

Скорость выполнения:
ONNX FP32: 68.36 мс/запрос
ONNX FP16: 100.72 мс/запрос
Ускорение: 0.68x